

ALaSegunda

Test Plan

Version 1.4

16/05/2026

UC2 – Purchase

Confidential

© ALaSegunda, 2025

Revisions

Date	Version	Description	Author
16/05/2026	1.4	Refactor purchase UI and backend tests (PR #188). Added PostPurchases_test, updated GetPurchase_test, added new UIT cases for cart validation and quantity modification. Removed DTO-only unit test claims. Updated milestones.	Héctor
14/12/2025	1.3	UIT for Select Products, Purchase Products and Details	Héctor
13/12/2025	1.2	UI for Purchase Details	Héctor
13/12/2025	1.1	UI for Select Products & Create Purchase	Héctor
11/12/2025	1.0	Fix Product Price DTO & Purchase Constructors	Héctor

Contents

Revisions	2
1 Test plan	5
1.1 Test plan identifier	5
2 Introduction	5
2.1 Summary of Software Items and Features	5
2.2 Need and History	5
3 Test items	6
3.1 Identification of Test Items	6
3.2 Transmittal Media & Requirements	6
3.3 References to Documentation	6
3.4 Exclusions	6
4 Features to be tested	7
4.1 UI: Select Products	7
4.2 UI: Create Purchase	7
4.3 UI: Purchase Details View	7
4.4 Backend Controller Logic	7
5 Features not to be tested	7
5.1 Delivery Assignment Details	7
5.2 Backend POST Method Internal Logic	7
6 Approach	8
6.1 Overall Strategy	8
6.2 Testing Activities by Feature Group	8
6.3 Comprehensiveness & Completion Criteria	8
6.4 Constraints	8
7 Item pass/fail criteria	8
7.1 Automated UIT Criteria	9
7.2 Functional Criteria	9
7.3 Backend Controller Criteria	9
7.4 Fail Condition	9
8 Suspension criteria and resumption requirements	9
8.1 Suspension Criteria	9
8.2 Resumption Requirements	10
9 Test deliverables	10
9.1 Documents	10
9.2 Data & Tools	10
10 Testing tasks	10
11 Environmental needs	11

12 Responsibilities	11
12.1 Management & Design	11
12.2 Execution & Resolution	11
12.3 Support & Environment	11
13 Staffing and training needs	11
13.1 Staffing Requirements	11
13.2 Training Options	12
14 Schedule	12
14.1 Test Milestones & Task Duration	12
15 Risks and contingencies	12
15.1 Risk: Missing UI Identifiers	12
15.2 Risk: DTO Equality Order-Dependency	12
15.3 Risk: Scope Overlap	13
16 Approvals	13

1. Test plan

To prescribe the scope, approach, resources, and schedule of the testing activities. To identify the items being tested, the features to be tested, the testing tasks to be performed, the personnel responsible for each task, and the risks associated with this plan.

1.1 Test plan identifier

Identifier: ISII-2526-GrupoI-ALaSegunda-TP-UC2-Purchase-2026-05-16

Version: 1.4

Date: 16/05/2026

Scope: UC2 – Purchase

2. Introduction

2.1 Summary of Software Items and Features

The purpose of this test plan is to validate the functionality, usability, and stability of the Purchase Management Module and its associated User Interface (UI). The testing scope covers the current state of the codebase as of PR #188 (Refactor purchase UI and backend tests).

Key features to be tested include:

- **Product Selection UI:** Verification of the interface for selecting products prior to purchase, including filtering by name and colour, adding to cart, and modifying quantities.
- **Purchase Creation:** Validation of the complete flow to create a new purchase, including form fields, validation errors, and backend integration.
- **Purchase Details View:** Verification of the UI displaying the details of a confirmed purchase.
- **Backend Controller Tests:** Unit tests for `PurchasesController.GetPurchase`, `CreatePurchase`, and `ProductsController.GetProductsForPurchasing`.
- **Data Integrity:** Validation of DTO equality and hash behavior for stable assertions in controller tests (PR #188).

2.2 Need and History

These features were developed to complete the “Purchase” user story, moving from backend definition to a fully interactive frontend. The codebase has been refactored in PR #188 to improve test stability, add new backend coverage, and expand UI automation.

History:

- Dec 11 – Dec 14, 2025: Initial DTO fixes (PR #166), UI implementation (PR #168, #170), and first UIT suite (PR #173).
- May 16, 2026: Refactor of purchase UI tests/page-objects and backend tests (PR #188). Includes new `PostPurchases_test`, reworked `GetPurchase_test`, updated seed data, and expanded UI automation coverage.

3. Test items

3.1 Identification of Test Items

The following software items are submitted for testing. The build version corresponds to the commits merged as of May 16, 2026.

Test ID	Item	Version / Revision	Description
TI-001	Product Selection UI	Build 16.05 (PR #188)	The interface component allowing users to browse, filter, and select products. Includes cart management (add, remove, increase/decrease quantity) and the UIT suite.
TI-002	Purchase Creation Flow	Build 16.05 (PR #188)	The logic and UI forms required to convert selected products into a Purchase entity, including form validation.
TI-003	Purchase Details UI	Build 16.05 (PR #188)	The view component that renders the read-only details of a finalized purchase.
TI-004	Backend Controller Tests	Build 16.05 (PR #188)	Unit tests for <code>PurchasesController</code> (<code>GetPurchase</code> , <code>CreatePurchase</code>) and <code>ProductsController</code> (<code>GetProductsForPurchasing</code>). Tests use DTO equality assertions and SQLite in-memory database.

3.2 Transmittal Media & Requirements

- **Media:** The items are deployed in the Development/Staging environment via the repository's CI/CD pipeline.
- **Prerequisites:** Testing requires the latest backend API to be running and accessible.

3.3 References to Documentation

- **Requirements Specification:** See GitHub project board (<https://github.com/orgs/ISII2526GroupIALaSegunda/projects/1>) and PR #188.
- **Design Specification:** See GitHub pull requests #188, #173, #170, #168.

3.4 Exclusions

- **Delivery Assignment UI:** Testing of “Delivery Assignment” details (Task 164) is explicitly excluded from this specific test session, as it falls under a different ownership (Alejandro).

4. Features to be tested

The following features and combinations will be validated to ensure the UI behaves as expected and integrates correctly with the updated data structures.

4.1 UI: Select Products

- **Description:** Verify that users can browse, filter by name/colour, and select products from the available list. Verify cart operations (add, remove, increase/decrease quantity).
- **Test Design Ref:** Based on the scope of the UIT suite in `UC_Purchase_UIT.cs`.

4.2 UI: Create Purchase

- **Description:** Verify the complete flow of creating a purchase using the selected products, including validation of required fields.
- **Test Design Ref:** Based on the implementation in `CreatePurchase_P0.cs` and `UC_Purchase_UIT.cs`.

4.3 UI: Purchase Details View

- **Description:** Verify that the details of a purchase are rendered correctly in the UI after creation or selection.
- **Test Design Ref:** Based on the implementation in `PurchaseDetail_P0.cs` and `UC_Purchase_UIT.cs`.

4.4 Backend Controller Logic

- **Description:** Verify that the controller endpoints behave correctly:
 - `GetPurchase`: Returns correct DTO for existing orders; returns `NotFound` for missing IDs.
 - `CreatePurchase`: Returns `CreatedAtAction` on success; returns `BadRequest` with correct error keys for invalid items, empty carts, insufficient stock, or non-existent products.
 - `GetProductsForPurchasing`: Returns in-stock products, supports filtering by colour and name substring, excludes out-of-stock items.
- **Test Design Ref:** Unit tests in `GetPurchase_test.cs`, `PostPurchases_test.cs`, and `GetProductsForPurchasing_test.cs` (PR #188).

5. Features not to be tested

The following features are explicitly excluded from this test plan as they fall under different development tasks or owners.

5.1 Delivery Assignment Details

- **Reason for Exclusion:** This feature is developed under Task 164 (seeing details of a delivery assignment) and is managed by a different author (Alejandro). It is outside the scope of the Purchase UI workflow.

5.2 Backend POST Method Internal Logic

- **Reason for Exclusion:** While the UI interacts with it, the internal logic and low-level unit testing of the POST method implementation are covered by the existing controller tests. This plan focuses on integration-level controller validation rather than mocking internals.

6. Approach

6.1 Overall Strategy

The testing approach adopts a bottom-up strategy, starting with data structure validation (DTOs) and moving to UI interaction and full workflow automation. The effort is divided into two major phases: Unit Testing for backend controllers and Automated User Interface Testing (UIT) for the visual workflows.

6.2 Testing Activities by Feature Group

Group A: Backend Controller Tests

- **Focus:** `PurchasesController` (`GetPurchase`, `CreatePurchase`) and `ProductsController` (`GetProductsForPurchasing`) (PR #188).
- **Technique:** Unit Testing with SQLite in-memory database and Moq for logging.
- **Activity:** Verify DTO equality assertions, correct HTTP status codes, and error key mapping for validation failures.

Group B: UI Components & Workflow (Purchase & Products)

- **Focus:** Select products list, Create Purchase form, and Purchase Details view (PR #188).
- **Technique:** Automated UI Testing (UIT) and Manual Visual Inspection.
- **Activity:**
 - **UIT Execution:** Run the test suite in `UC_Purchase_UIT.cs`.
 - **Validation:** Ensure the UI navigates correctly from product selection to the purchase creation form, handles empty carts, filters products, validates form fields, and displays purchase details accurately.

6.3 Comprehensiveness & Completion Criteria

- **Code Coverage:** Controller unit tests must cover success paths and all documented error branches (empty items, non-existent product, insufficient stock).
- **Requirement Coverage:** 100% of the acceptance criteria defined in the UIT suite must be mapped to at least one test case.
- **Traceability:** Test cases are traced back to the specific Pull Request ID (#188) to ensure every deployed change has a corresponding verification step.

6.4 Constraints

- **Backend Dependency:** The UI testing for “Create Purchase” is strictly blocked by the availability and stability of the backend API.
- **Timeline:** All testing must be completed within the sprint window, coordinating with the merge dates of the PRs.

7. Item pass/fail criteria

A test item will be considered PASSED only if it meets the following criteria:

7.1 Automated UIT Criteria

- The automated test suite (`UC_Purchase_UIT.cs`) runs to completion without errors or time-outs.
- The UI correctly identifies and interacts with the products list, cart controls, and form fields.

7.2 Functional Criteria

- **Create Purchase:** The system successfully generates a Purchase ID upon valid form submission.
- **Details View:** The “Purchase Details” view accurately reflects the data entered during creation, including the correct product prices.
- **Cart Operations:** Increasing/decreasing quantity and removing items from the cart update the UI correctly.
- **Validation:** Submitting the form with empty required fields displays the appropriate validation messages.

7.3 Backend Controller Criteria

- `GetPurchase` returns the expected `PurchaseForDetailDTO` for existing orders and `NotFoundResult` for missing IDs.
- `CreatePurchase` returns `CreatedAtActionResult` with a valid DTO on success, and `BadRequestObjectResult` with the correct error keys (“Items”, “Stock”) for invalid input.
- `GetProductsForPurchasing` returns only in-stock products and correctly applies colour and name filters.

7.4 Fail Condition

Any “Blocker” or “Critical” severity bug found during the test execution of the “Select Product” or “Create Purchase” flows constitutes an immediate FAIL for the respective item.

8. Suspension criteria and resumption requirements

8.1 Suspension Criteria

Testing activities on the Purchase UI and logic will be suspended immediately under the following conditions:

- **Backend API Failure:** If the backend returns consistent 500 Internal Server Errors or is unavailable, blocking the “Create Purchase” flow.
- **Critical Blocking Defects:** Any defect that causes the application to crash upon opening the “Select Products” view or prevents the user from navigating to the “Purchase Details” view.
- **Data Structure Mismatch:** If the updated DTOs (PR #188) fail to map fields correctly, rendering the test data invalid.

8.2 Resumption Requirements

- **Fix Verification:** Testing will resume only after the blocking issue is resolved and a new build is deployed to the test environment.
- **Regression Activity:**
 - If the suspension was due to API/DTO issues, the Unit Tests for controllers (PR #188) must be re-run first.
 - If the suspension was due to UI crashes, the specific UIT suite must be re-executed from the beginning to ensure stability.

9. Test deliverables

The following documents and artifacts are to be delivered upon completion of the testing process:

9.1 Documents

- **Test Plan:** This current document outlining the strategy for the Purchase Module (v1.4).
- **Test Design & Case Specifications:** The source code and scripts for the Automated UI Tests (UIT) and unit tests (PR #188).
- **Test Logs:** Execution logs generated by the CI/CD pipeline during the run of PR #188.
- **Test Incident Reports:** GitHub Issues created for any bugs found during manual inspection.

9.2 Data & Tools

- **Test Input Data:** JSON datasets and SQLite seed scripts used to mock the “Select Products” list and purchase data.
- **Test Tools:** The test automation framework (Selenium WebDriver for UIT, xUnit for unit tests) configured in the project.

10. Testing tasks

The following tasks are required to prepare for and perform the testing of the Purchase features.

Task 1: Environment Preparation

- **Description:** Ensure the test environment has the latest backend code deployed (PR #188).

Task 2: Backend Controller Verification

- **Description:** Verify that `GetPurchase`, `CreatePurchase`, and `GetProductsForPurchasing` controller tests pass.
- **Resources:** PR #188 (Refactor purchase UI and backend tests).
- **Date:** To be completed at the start of the test cycle.

Task 3: UI Functionality Testing (Manual/Exploratory)

- **Description:** Manually verify the “Purchase Details” view, “Create Purchase” form, and cart operations.
- **Resources:** PR #188.

- **Dependencies:** Task 2 must be complete.

Task 4: Automated UIT Development & Execution

- **Description:** Run the automated UI tests for the product selection and purchase workflow.
- **Resources:** UC_Purchase_UIT.cs (PR #188).
- **Skills:** UI Automation scripting.

Task 5: Reporting

- **Description:** Compile pass/fail results from the UIT and unit tests and close the associated Pull Requests.
- **Dependency:** Task 4 must be successful.

11. Environmental needs

- **Test environment:** Backend API deployed and reachable (PR #188), frontend build including the updated purchase pages, and Chrome/Chromium for UIT execution.
- **Tools:** Unit test runner (xUnit), UIT framework (Selenium WebDriver), and CI/CD logs for evidence.
- **Test data and access:** Seeded products list with valid stock levels and a test user account allowed to create and view purchases.

12. Responsibilities

The following groups and individuals are responsible for the management, execution, and support of the testing activities defined in this plan:

12.1 Management & Design

- **Test Lead / Designer:** Héctor is responsible for designing the Test Cases, creating the Test Plan, and developing the Automated UI Tests (UIT) for the Purchase workflow.

12.2 Execution & Resolution

- **Test Executor:** Héctor will execute the automated UIT suite and perform manual validation of the Purchase UI.

12.3 Support & Environment

- **Configuration Support:** Héctor is responsible for ensuring the test environment is prepared with the necessary HTML IDs on form fields to facilitate automation.
- **Excluded Items Owner:** Alejandro is responsible for the Delivery Assignment module, ensuring its development does not negatively impact the shared environment.

13. Staffing and training needs

13.1 Staffing Requirements

- **Test Engineer:** Required skill level includes proficiency in C# (for controller unit tests) and UI Automation frameworks (Selenium for UIT). This role is currently filled by the feature developer (Héctor).

13.2 Training Options

- While the staff is familiar with the general project architecture, specific training was required for the Automated UI Testing (UIT) framework. The Test Engineer (Héctor) dedicated a self-study period to acquire the necessary skills to implement the automation scripts for the “Select Products” and “Purchase” workflows, ensuring the successful execution of the test suite.

14. Schedule

The testing schedule is aligned with the development sprint activities.

14.1 Test Milestones & Task Duration

Milestone / Task	Start Date	End Date	Resource / Owner
M1: Backend Controller Validation	16/05/2026	16/05/2026	Héctor
	Validation of <code>GetPurchase</code> , <code>CreatePurchase</code> , and <code>GetProductsForPurchasing</code> controller tests (PR #188).		
M2: UIT Execution	16/05/2026	16/05/2026	Héctor
	Execution of the full UIT suite including cart operations, filtering, form validation, and purchase details (PR #188).		
M3: Reporting & Sign-off	16/05/2026	16/05/2026	Héctor
	Compilation of pass/fail results and closure of PR #188.		

15. Risks and contingencies

The following high-risk assumptions have been identified for this test plan:

15.1 Risk: Missing UI Identifiers

- **Description:** The automated tests rely on specific IDs in the HTML (e.g., `btnAdd_Name`, `btnIncrease_Name`). If product names contain spaces or special characters, these IDs may be invalid or unstable, causing UIT failures.
- **Mitigation:** Ensure element IDs use stable identifiers such as `ProductId` rather than product names.

15.2 Risk: DTO Equality Order-Dependency

- **Description:** `PurchaseForDetailDTO.Equals` uses `Items.SequenceEqual`, making controller tests order-dependent. EF Core does not guarantee item ordering without an explicit `OrderBy`.
- **Mitigation:** Consider sorting items by `ProductId` in the controller projection or using an order-insensitive comparison in `Equals`.

15.3 Risk: Scope Overlap

- **Description:** Simultaneous modification of Product DTOs or seed data might affect the “Delivery Assignment” tests running in parallel.
- **Mitigation:** Coordinate seed data changes with the Delivery Assignment test owner (Alejandro).

16. Approvals

The following persons must approve this plan before the final release of the software version.

Name	Title	Signature	Date
Héctor Ruiz	Lead Developer / Test Author	_____	16/05/2026
Alejandro Gómez	Project Manager	_____	___/___/_____
Elena María	Project Supervisor	_____	___/___/_____